

A Code, Prompts, Trajectories

All code is available at <https://github.com/princeton-nlp/tree-of-thought-llm>.

All prompts are available at <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/src/tot/prompts>.

Trajectories are available at <https://github.com/princeton-nlp/tree-of-thought-llm/tree/master/logs>.

B Additional Experiment Results

Given the motivation of exploring and extending the capability frontier of language models, our experiments in the main paper have focused on a setup with the state-of-the-art language model (GPT-4), and three hard tasks invented to challenge it. Here, we report additional experiments with weaker LLM or easier tasks, and discuss cost and efficiency.

	GSM8K	StrategyQA		GPT-4	GPT-3.5		GPT-4	GPT-3.5
IO	51	73	IO	7.3%	6%	IO	6.19	4.47
CoT	86	82	CoT	4.0%	3%	CoT	6.93	5.16
ToT	90	83	ToT	74%	19%	ToT	7.56	6.62

Table 4: New tasks with zero-shot ToT and GPT-4.

Table 5: Game of 24 with GPT-4 vs GPT-3.5.

Table 6: Creative Writing with GPT-4 vs. GPT-3.5.

B.1 Extension to new tasks (GSM8k, StrategyQA) with zero-shot ToT

While more common NLP tasks might be too easy for GPT-4 and do not require ToT (which is why we considered harder new tasks), we believe applying ToT to new tasks could be straightforward. For example, we implemented a simple and generic zero-shot ToT-BFS similar to creative writing (sample 5 problem solving strategies then vote for the best one; then sample 5 solutions based on the best strategy then vote for the best one) for GSM8K and StrategyQA with few extra lines of code:

```
# define the answer format of new tasks
gsm8k_format = "the answer is n" where n is a number'
strategyqa_format = 'either "the answer is yes" or "the answer is no"'

# define zero-shot io prompting
standard_prompt = 'Answer the following question with {format}: {input}'

# define thought format for zero-shot cot and zero-shot tot
cot_prompt = "'Answer the following question: {input}'

Make a strategy then write. Your output should be of the following format:

Strategy:
Your strategy about how to answer the question.

Answer:
Your answer to the question. It should end with {format}.'
''',

# define zero-shot voting used for zero-shot tot
vote_prompt = "'Given an instruction and several choices,
decide which choice is most promising.
Analyze each choice in detail, then conclude in the last line
"The best choice is {s}", where s the integer id of the choice.
''',
```

We evaluated on a subset of 100 random GSM8K test and StrategyQA dev questions. As shown in Table 4 and as expected, ToT improves over CoT on both tasks (but only slightly, given GPT-4 + CoT is already very good on such tasks, and StrategyQA’s bottleneck is external knowledge, not reasoning). Considering computational costs, it is more suitable to try smaller LLMs + ToT for traditional NLP tasks, or GPT-4 + ToT for hard tasks that challenge GPT-4 + CoT’s reasoning.

B.2 Extension to new LMs (GPT-3.5)

To understand how ToT works with other LLMs, we also ran GPT-3.5-turbo for Creative Writing (Table 6) and Game of 24 (Table 5). On both tasks, “ToT > CoT > IO” remains true for GPT-3.5. On Creative Writing, we find GPT-3.5+ToT outperform GPT-4+IO, and similar to GPT-4+CoT, which suggests ToT could also work well on weaker language models.

On Game of 24 (we changed 1-shot proposal prompt to 3-shot to make it work), GPT-3.5+ToT’s 19% is far worse than GPT-4+ToT’s 74%. To further understand the importance of generation vs. evaluation, we ran GPT-4 generation + GPT-3.5 evaluation (64%) and GPT-3.5 generation + GPT-4 evaluation (31%). This suggests the game’s bottleneck is thought generation, and different generation/evaluation language models might attain decent results while reducing costs.

B.3 Cost and efficiency

Running ToT requires significantly more computations than IO or CoT prompting. For example, in Game of 24 (Table 7 below), solving a problem with ToT requires 5.5k completion tokens, close to 100 CoT trials (6.7k tokens). But the performance of ToT is better than best of 100 independent CoT trials.

Game of 24	Generate/Prompt tokens	Cost per case	Success
IO (best of 100)	1.8k / 1.0k	\$0.13	33%
CoT (best of 100)	6.7k / 2.2k	\$0.47	49%
ToT	5.5k / 1.4k	\$0.74	74%

Table 7: Cost analysis on Game of 24.

On Creative Writing (Table 8 below), we found ToT takes around 5x completion tokens and money cost, which is intuitive as $b = 5$ and most tokens are generated passages.

Creative Writing	Generate/Prompt tokens	Cost per case
IO	0.9k / 0.4k	\$0.06
CoT	0.9k / 0.4k	\$0.07
ToT	4k / 2.9k	\$0.32

Table 8: Cost analysis on Game of 24.

So completing Game of 24 and Creative Writing’s main ToT experiments cost around $0.74 \times 100 + 0.32 \times 100 = 106$ dollars. Crosswords’ DFS experiments should be also within 100 dollars. In general, cost and efficiency of ToT highly depend on the prompts and search algorithms used, and could require 5-100 times more generated tokens than CoT. Some actionable insights:

- We recommend using ToT on tasks requiring deliberate reasoning, on which CoT struggles.
- Flexibility of ToT allows some performance-cost tradeoff, e.g., change beam size or vote number in BFS, few-shot vs. zero-shot prompting, GPT-3.5 vs. GPT-4, etc. One could configure the setup based on some resource constraints or performance goal.
- There is much space for improving efficiency, e.g., BFS could early stop when solution is found, or trim down beam size to when some thoughts are “impossible”.
- We believe that more computation is indeed required in order for the model to achieve stronger intelligence, and this should not become a blocking issue as in the long run, (open-source) LMs will become much cheaper and more efficient. It is also a great direction how to better train/finetune LMs for thought generation and/or evaluation.